

IMPLEMENTATION BLUEPRINT

Automated Outbound Calling & IVR Platform

Detailed development plan, delivery sequence, functional scope, UI plan, and acceptance gates for Auckland Accounting Services Ltd.

Source of truth

Acula-new.pdf - approved stack and functional requirements

Prepared

3 September 2026

Purpose

Keep this document with the repository and use it as the delivery checklist throughout development.

Status: Approved roadmap for implementation. This plan deliberately treats the existing React application as a UI prototype, not as production telephony software.

1. Executive summary

The current project is a high-quality frontend demonstration of an IVR operations panel. It contains useful visual screens and a browser-based call simulator, but all data is browser-local and no real call is placed. The objective is therefore to retain the useful interface while replacing its demo-only state and simulated behavior with a secure API, PostgreSQL database, Redis/BullMQ workers, and Twilio Voice integration.

The build will follow the approved technology stack: React, Node.js/Express, PostgreSQL, Prisma, Redis, BullMQ, Twilio Voice, Nginx, PM2, Ubuntu 24.04 LTS, HTTPS and REST APIs.

Delivery principle

- No outbound call may be sent until authentication, DNC suppression, calling-hours rules, pre-launch validation, audit records and Twilio configuration are operational.
- The existing dashboard, contacts, campaigns, IVR builder, logs and analytics become API-driven screens rather than being discarded.
- The MVP uses deterministic IVR and DTMF. Speech/AI features remain optional future modules, as required by the specification.

2. Current-state baseline

Area	Present today	Required outcome
Frontend	React single-page interface with local state and localStorage.	Route-based, authenticated React frontend consuming REST APIs.
Data	Static seed data and browser-local changes.	PostgreSQL records, Prisma migrations, transactional writes and backups.
Calling	Browser simulator and generated sample TwiML only.	Twilio outbound calls, live webhooks, DTMF capture and persisted provider IDs.
Campaign processing	Three-contact random simulation.	BullMQ scheduling, worker concurrency, retries, pause/resume and stop conditions.
Security	No users, authentication or authorization.	JWT, refresh tokens, password hashing, RBAC, validation, rate limits and HTTPS.
Reporting	Frontend calculations based on demo logs.	Server-side filtered reports and CSV/Excel exports.

Immediate remediation items

- Remove all production-looking hard-coded credentials, example API keys, provider URLs and telephone identities from the frontend.
- Standardize seed copy and sample data to Auckland Accounting; remove mixed provider references and non-NZ placeholder identities before UAT.
- Treat the present audit trail as a visual mockup until immutable server-side audit logging is installed.

3. Target architecture

The production system consists of an admin browser, React frontend, Nginx reverse proxy, Express REST API, PostgreSQL/Prisma, Redis/BullMQ workers and Twilio Voice. The API must remain responsive; campaign scheduling and calling execute only in worker processes.

Layer	Responsibilities	Key safeguards
React admin panel	Authenticated operations UI, forms, flow preview, reports and polling.	Route guard, form validation, no secrets in client.

Layer	Responsibilities	Key safeguards
Express API	REST APIs, authorization, campaign validation, webhook endpoints and reporting.	Zod validation, rate limits, CORS, Helmet, centralized errors.
PostgreSQL + Prisma	Contacts, campaigns, call jobs/attempts/responses, users, audit data.	Foreign keys, indexes, migrations, transactions and backups.
Redis + BullMQ	Scheduling, outbound jobs, retries, reports and notifications.	Idempotent job IDs, controlled concurrency and failed-job alerts.
Twilio Voice	Outbound dialing, Gather/DTMF, status callbacks and optional recording.	Signature validation, provider call ID mapping and retry classification.
Nginx + PM2	HTTPS, static app, API proxy and resilient service processes.	TLS renewal, request limits, health checks and restart policy.

4. Delivery phases

Phase 0 - Product alignment and repository preparation

Confirm roles, calling policy, DNC/consent rules, Twilio account ownership, phone number, calling windows, retention periods, hosts and environments. Record final field definitions and acceptance scenarios. Clean prototype copy and establish branch, code-review and release conventions.

Phase 1 - Foundation and data model

Introduce Express API structure, PostgreSQL, Prisma schema/migrations, Redis/BullMQ, Docker local services, environment validation and API documentation. Establish users, roles, permissions, contacts, groups, campaigns, questions, options, jobs, attempts, responses, events, audit logs and settings.

Phase 2 - Authentication and operational security

Implement login/logout, refresh tokens, password reset, activation/deactivation, bcrypt/Argon2 hashing and role-based authorization. Add Zod, Helmet, CORS, rate limits, structured logs, error responses and secret-handling conventions.

Phase 3 - Contacts, consent and import workflow

Move contacts to API CRUD. Add contact groups, tags, valid E.164 number normalization, consent source/timestamp, DNC suppression, duplicate detection and contact history. Build the CSV mapping, validation, preview, confirmation, transaction and export workflows.

Phase 4 - Campaign and questionnaire engine

Implement campaign CRUD, lifecycle states, schedules, time zones, business hours, blackout periods, retry policies, concurrency, daily limits and stop conditions. Persist reusable versioned questionnaires, question types, options and actions. Add pre-launch validation and a visual flow graph.

Phase 5 - Queue processing and Twilio Voice

Build scheduler/call/retry workers, job creation, pacing and campaign pause/cancel behavior. Integrate Twilio outbound calls and server-generated TwiML. Implement signed answer, input, status and recording webhooks; safely save all attempts, responses and events.

Phase 6 - Reporting and live operations

Replace demo metrics with queries and polling. Deliver campaign, call, contact and response reports; export CSV/Excel; operational queue metrics; live campaign detail; global emergency stop and system health visibility.

Phase 7 - Hardening, UAT and deployment

Complete unit/integration/E2E/failure testing. Deploy Ubuntu services via Nginx and PM2 with HTTPS, backups, monitoring, alerting and recovery playbooks. Run UAT with approved Twilio test numbers before production launch.

5. Data model and API plan

Core tables

Domain	Primary tables	Notes
Identity	users, roles, permissions, user_roles, refresh_tokens	Role and action checks occur server-side.
Contacts	contacts, contact_groups, contact_group_members, imports	Store consent/DNC, normalized phone, source and tags.
Campaigns	campaigns, campaign_contacts, campaign_runs	Keep target membership and eligibility snapshots.
IVR	questionnaires, question_versions, questions, question_options	Options store next question or action: CONTINUE, END_CALL, TRANSFER, SKIP.
Telephony	call_jobs, call_attempts, call_responses, call_events, retry_logs	Provider call ID and provider event IDs must be unique where applicable.
Governance	audit_logs, system_settings, notifications	Audit entries include actor, entity, old/new value, IP and user agent.

REST API surface

/api/auth, /api/users, /api/roles, /api/contacts, /api/contact-groups, /api/campaigns, /api/questionnaires, /api/call-jobs, /api/call-responses, /api/reports, /api/audit-logs, /api/settings, and /api/webhooks/twilio.

Every request uses a consistent response shape, pagination where appropriate, validation errors without stack traces, and authorization checks matched to the user role. Webhooks are deliberately isolated from admin APIs and must verify the Twilio signature before processing.

6. Functional workflow specifications

Contact import

Upload -> map columns -> validate required fields -> normalize phone numbers -> detect duplicates -> show valid/invalid/duplicate preview -> user confirms -> transactional insert -> show final import report. No import may silently create records.

Campaign launch

Admin selects a contact group or filtered set, questionnaire version, calling window/time zone, retry policy, daily cap, maximum concurrency, caller ID and dates. The system validates every dependency before changing status from DRAFT/SCHEDULED to RUNNING. A launch event and job batch are recorded atomically.

Call execution

The scheduler finds eligible contacts within their permitted calling window and creates idempotent jobs. A worker asks Twilio to call. Twilio requests answer TwiML, gathers DTMF, posts inputs and status callbacks, then the API records events/responses and determines the next question/action. Retryable outcomes create a delayed retry job; permanently invalid numbers are never retried.

Idempotency and failure handling

- Use unique provider event identifiers and database transactions to avoid duplicate responses, status changes and retry jobs.
- Classify busy, no-answer, temporary provider failure and permanent failure separately.
- Campaign pause prevents new dispatches; cancel prevents new dispatches and clears eligible queued work without destroying historical attempts.

- All worker and provider failures must be observable through logs, queue dashboards and alerts.

7. UI implementation plan

Page	Main functions	Completion condition
Login / recovery	Login, forgot/reset password, session expiry.	Unauthenticated users cannot access app data.
Dashboard	Live KPIs, active calls, queue state, warnings and emergency stop.	Values come from API; clear stale-data state is shown.
Contacts	Directory, profile, groups, DNC, consent, filters and import/export.	Server-side pagination, validation and full history work.
Campaigns	List, create wizard, details, edit, start/pause/resume/stop/export.	Launch cannot proceed on invalid configuration.
Question builder	Draft/publish versions, drag/drop, options, branches, preview and validation graph.	Invalid destinations and cycles are explained before launch.
Calls	Queue, live progress, attempts, events, responses, transcript and optional recording.	Details reconcile with provider callbacks.
Reports	Campaign, response and contact reports with filters and exports.	Exports match filtered server results.
Administration	Users, roles, settings, Twilio status, calling rules, retention and audit.	Permissions visibly and server-side enforce access.

Design rules

- Retain the current operational visual style, but use consistent empty, loading, error and permission-denied states.
- Use clear destructive-action confirmations for stop, cancel, delete and DNC changes.
- Display server status honestly: “simulator,” “test mode,” “queued,” “in progress,” and “provider unavailable” must never be visually confused.
- Keep personal and financial details masked by default where the role does not require them.

8. Testing and release gates

Test layer	Coverage required	Gate
Unit	Validation, flow routing, DNC eligibility, retry decision, role checks.	Required on every pull request.
Integration	Prisma transactions, Redis/BullMQ, API endpoints, Twilio signatures and duplicate callbacks.	Required before staging.
End-to-end	Contact import -> campaign -> queue -> test call -> DTMF -> response -> report.	Required before UAT.
Failure testing	Provider outage, invalid number, busy/no answer, duplicate webhook, DB/Redis outage, worker restart, pause/cancel mid-run.	Required before production.
Security	Authentication, access control, input validation, secrets, HTTPS and dependency scan.	Required before production.
UAT	Approved users execute acceptance scenarios with test numbers.	Written approval before production calls.

MVP acceptance checklist

- Admin signs in securely and roles restrict access.
- Contacts and groups can be imported, validated, deduplicated, exported and DNC-suppressed.
- Admin can build validated branching IVR flows and schedule campaigns.
- BullMQ dispatches only eligible calls, respects concurrency and calling windows, and Twilio captures DTMF.

- Responses, attempts, events, audits and reports persist in PostgreSQL.
- Retries behave as configured; duplicate webhooks do not duplicate records.
- Campaign pause/resume/stop and global emergency stop work reliably.
- Production HTTPS, backups, monitoring and deployment documentation are complete.

9. Deployment and operating model

Production will run on Ubuntu 24.04 LTS with Nginx handling HTTPS/static assets/API proxying and PM2 operating API, scheduler and call-worker processes. PostgreSQL and Redis must have access controls, routine patching and monitored backups. Large uploads and call recordings use object storage; the database stores keys and metadata only.

Required operational deliverables

- Environment-variable template and secret rotation procedure.
- Prisma migrations, seed data and rollback process.
- Nginx/PM2 deployment instructions and health endpoints.
- Daily database backups, verification, retention and restoration procedure.
- Monitoring for API, workers, queues, database, Redis, disk, SSL expiry and Twilio webhook errors.
- Administrator guide covering users, imports, DNC, campaigns, question flows, reports and emergency stop.
- API and webhook documentation plus test evidence.

Future modules explicitly deferred from the deterministic-IVR MVP

AI voice agent, natural-language speech processing, AI summaries, SMS/WhatsApp delivery, CRM integrations, multi-tenant management, sentiment analysis and lead scoring remain extension points. They may be planned only after the core call engine, security and data controls pass UAT.

10. First implementation sprint

The first sprint establishes the base that every later feature depends on. It should not place real calls.

Work item	Deliverable	Verification
Project structure	Separate frontend and API folders, shared types/contracts, lint/test scripts.	Both apps start locally and CI runs checks.
Local services	Docker Compose or documented local PostgreSQL and Redis services.	Health checks and clean boot sequence.
Prisma foundation	Schema, initial migration, seed process and database client.	Migration applies from an empty database.
Identity	User, role, permission schema and protected API skeleton.	Protected endpoint rejects unauthenticated and unauthorized requests.
Configuration	Validated environment template without committed secrets.	Startup fails safely when required configuration is absent.
Observability	Request IDs, structured logs and central error middleware.	Errors return safe responses and logs contain no secrets.
Frontend connection	Router, authenticated API client, query provider and app shell.	Existing visual dashboard can read a protected health/current-user endpoint.

Decision log to confirm before real calling

- Named owner for Twilio account, billing, phone number and geographic permissions.
- Approved calling policy: timezone, business hours, holidays, blackout dates and maximum daily/campaign spend.
- Client-approved consent wording, DNC process, privacy retention periods and recording policy.
- Production hosting, domain, notification recipients, backup retention and incident contacts.
- Final user roles and which staff can start, pause, cancel or export campaigns.

End of plan - use this as the baseline checklist for implementation reviews and UAT sign-off.